

Proyecto 2

Integrantes del equipo:

Isabella Cadavid Posada

Isabella Ocampo Sánchez

Profesor:

José Luis Montoya Pareja

Materia:

Organización de computadores

Universidad EAFIT
Escuela de Ciencias Aplicadas e Ingeniería
Medellín
2026

Tabla de Contenido

Videos explicativos de cada chip.....	4
Chip Shifter.....	5
1. Descripción General.....	5
2. Interfaz del Componente.....	5
3. Descripción Funcional.....	5
3.1 Desplazamiento a la izquierda (direction = 0).....	5
Definición formal:.....	6
3.2 Desplazamiento a la derecha (direction = 1).....	6
Definición formal:.....	6
4. Implementación del Circuito.....	6
4.1 Uso de Multiplexores.....	6
4.2 Construcción de señales internas.....	7
Desplazamiento a la izquierda.....	7
Desplazamiento a la derecha.....	7
4.3 Selección final.....	7
4.4 Cálculo de la salida result.....	7
5. Casos de Prueba.....	7
6. Video explicativo.....	9
Unidad Aritmético Lógica (ALU) con Shifter Integrado.....	10
1. Descripción General.....	10
2. Interfaz del Componente.....	10
3. Descripción Funcional e Implementación del Circuito.....	11
3.1 Preprocesamiento de entradas.....	12
3.2 Operaciones aritmético-lógicas.....	12
3.3 Modo Shifter integrado.....	12
Activación del modo Shifter.....	13
3.4 Operaciones de desplazamiento.....	13
Desplazamiento a la izquierda:.....	13
Desplazamiento a la derecha:.....	13
3.5 Selección final de salida.....	13
3.6 Cálculo de flags.....	14
Flag negativo (ng) -> Se obtiene directamente del bit más significativo: ng = out[15]... 14	
Flag cero (zr) -> Se calcula verificando si todos los bits de salida son cero:.....	14
3.7 Cálculo de la salida result.....	14
4. Casos de prueba.....	15
5. Video Explicativo.....	16
Arquitectura de la Instrucción para Operaciones Shift.....	17
1.1 Formato de la instrucción tipo C.....	17
1.2 Problema identificado.....	17

1.3 Solución propuesta.....	17
Reglas de codificación:.....	17
1.4 Traducción de instrucciones.....	18
1.5 Ejemplos.....	19
1.6 Conclusión.....	19
Memoria.....	20
1. Descripción General.....	20
2. Interfaz del Componente.....	20
3. Descripción Funcional e Implementación.....	20
3.1 Combinaciones.....	21
3.2 Proceso de Escritura (DMux4Way).....	21
3.3 Proceso de Lectura (Mux4Way16).....	21
4. Pruebas.....	21
5. Video Explicativo.....	22
Unidad Central de Procesamiento (CPU).....	23
1. Descripción General.....	23
2. Interfaz del Componente.....	23
3. Descripción Funcional e Implementación.....	24
4. Pruebas.....	25
5. Video Explicativo.....	26
Computador.....	27
1. Descripción General.....	27
2. Interfaz del Componente.....	27
3. Descripción Funcional e Implementación.....	27
4. Pruebas.....	28
5. Video Explicativo.....	30

Videos explicativos de cada chip

- **SHIFTER:** <https://youtu.be/Q1Jp9Q8Ld4Q?feature=shared>
- **ALU:** <https://youtu.be/OEYjY9LSjQg>
- **MEMORIA:** <https://www.youtube.com/watch?v=mq-yS4MR1L0&t=411s>
- **CPU:** <https://www.youtube.com/watch?v=NrBvYcYYwWE>
- **COMPUTADOR:** <https://www.youtube.com/watch?v=8Qnr-G1wdPo>

Chip Shifter

1. Descripción General

El chip Shifter realiza operaciones lógicas de desplazamiento (shift) de un bit sobre un bus de entrada de 16 bits (in). El sentido del desplazamiento se controla mediante un bit selector de dirección (direction). El componente retorna el bus resultante desplazado (out) y aísla el bit que es expulsado del bus durante la operación (result).

2. Interfaz del Componente

Puerto	Ancho	Dirección	Descripción
in	16 bits	Entrada	Bus de datos original a desplazar.
direction	1 bit	Entrada	Selector de operación lógica: (Desplazamiento a la Izquierda) 0 (Desplazamiento a la Derecha) 1
out	16 bits	Salida	Bus de datos tras aplicar el desplazamiento.
result	1 bit	Salida	Bit desplazado fuera del rango de 16 bits.

3. Descripción Funcional

El comportamiento del sistema depende del valor de direction.

3.1 Desplazamiento a la izquierda (direction = 0)

- Cada bit se mueve una posición hacia la izquierda
- Se inserta un **0** en el bit menos significativo
- El bit más significativo se pierde y se guarda en **result**

Definición formal:

$out[0] = 0$

$out[i] = in[i-1]$ para $i = 1..15$

$result = in[15]$

3.2 Desplazamiento a la derecha (direction = 1)

- Cada bit se mueve una posición hacia la derecha
- Se inserta un **0** en el bit más significativo
- El bit menos significativo se pierde y se guarda en **result**

Definición formal:

$out[15] = 0$

$out[i] = in[i+1]$ para $i = 0..14$

$result = in[0]$

4. Implementación del Circuito

4.1 Uso de Multiplexores

Cada bit de salida se calcula con un **Mux de 2 entradas**:

$out[i] = Mux(a = leftShift[i], b = rightShift[i], sel = direction)$

Donde:

- **a** corresponde al desplazamiento a la izquierda
- **b** corresponde al desplazamiento a la derecha
- **sel** determina cuál operación usar

4.2 Construcción de señales internas

Desplazamiento a la izquierda

- $\text{leftShift}[0] = 0$
- $\text{leftShift}[i] = \text{in}[i-1]$ para $i = 1..15$

Desplazamiento a la derecha

- $\text{rightShift}[15] = 0$
- $\text{rightShift}[i] = \text{in}[i+1]$ para $i = 0..14$

4.3 Selección final

Se implementan **16 Mux independientes**, uno por cada bit:

Ejemplo:

$\text{Mux}(a=\text{false}, b=\text{in}[1], \text{sel}=\text{direction}, \text{out}=\text{out}[0]);$

$\text{Mux}(a=\text{in}[0], b=\text{in}[2], \text{sel}=\text{direction}, \text{out}=\text{out}[1]);$

...

$\text{Mux}(a=\text{in}[14], b=\text{false}, \text{sel}=\text{direction}, \text{out}=\text{out}[15]);$

4.4 Cálculo de la salida result

Se utiliza un Mux adicional:

$\text{result} = \text{Mux}(a = \text{in}[15], b = \text{in}[0], \text{sel} = \text{direction})$

Interpretación:

- Si $\text{direction} = 0$ -> sale el bit más significativo
- Si $\text{direction} = 1$ -> sale el bit menos significativo

5. Casos de Prueba

Para ilustrar el comportamiento de forma concisa, se presenta la siguiente tabla de verdad.

Operación	Entrada direction	Entrada in[16]	Salida out[16]	Salida result
Shift Izquierda	0	1000 0000 0000 0001	0000 0000 0000 0010	1
Shift Derecha	1	1000 0000 0000 0001	0100 0000 0000 0000	1
Shift Izquierda	0	0011 1100 0011 1100	0111 1000 0111 1000	0
Shift Derecha	1	0011 1100 0011 1100	0001 1110 0001 1110	0

Hardware Simulator (2.5) - C:\Users\Usuario\OneDrive\Documents\Isabella\4_SEMESTRE\Organizacion_de_computadores\Pruebas del proyecto 2\Shifter.hdl

File View Run Help

Chip Name: Shifter Time: 0

Input pins		Output pins	
Name	Value	Name	Value
in[16]	-32767	out[16]	2
direction	0	result	1

HDL

```

/*****
 * Shifter.hdl - Desplazamiento
 * Autor 1: Isabella Cadavid P
 * Autor 2: Isabella Ocampo Sá
 */

load Shifter.hdl,
output-file Shifter.out,
compare-to Shifter.cmp,
output-list in%B1.16.1 direction%B3.1.3 out%B1.16.1 result%B3.1.3;

// CASO 1:
// direction = 0 → corrimiento a la IZQUIERDA
// in = 1000 0000 0000 0001 (0x8001 = 32769 decimal)
// out = 0000 0000 0000 0010 (0x0002 = 2 decimal)
// result = 1 (in[15] = 1, el Bit Más significativo sale por la izquierda)
//

set in %B10000000000000001,
set direction 0,
eval,
output;

// CASO 2:
// direction = 1 → corrimiento a la DERECHA
// in = 1000 0000 0000 0001 (0x8001 = 32769 decimal)
// out = 0100 0000 0000 0000 (0x4000 = 16384 decimal)
// result = 1 (in[0] = 1, el Bit Menos significativo sale por la derecha)
//

```

Internal pins

Name	Value
------	-------

Hardware Simulator (2.5) - C:\Users\Usuario\OneDrive\Documents\Usabella\4_SEMESTRE\Organizacion_de_computadores\Pruebas del proyecto 2\Shifter.hdl

File View Run Help

Chip Name: **Shifter** Time: 0

Input pins		Output pins	
Name	Value	Name	Value
in[16]	-32767	out[16]	16384
direction	1	result	1

HDL

```

/*****
 * Shifter.hdl - Desplazamiento
 * Autor 1: Isabella Cadavid P
 * Autor 2: Isabella Ocampo Sá
 *
 * Implementación del siguiente
 * if (direction == 0){
 *   out[0] = 0;
 *   for(i=1; i < 16; i++)

```

Internal pins

Name	Value
------	-------

```

load Shifter.hdl,
output-file Shifter.out,
compare-to Shifter.comp,
output-list in%B1.16.1 direction%B3.1.3 out%B1.16.1 result%B3.1.3;

//
// CASO 1:
// direction = 0 -> corrimiento a la IZQUIERDA
// in = 1000 0000 0000 0001 (0x8001 = 32769 decimal)
// out = 0000 0000 0000 0010 (0x0002 = 2 decimal)
// result = 1 (in[15] = 1, el Bit Más significativo sale por la i
//

set in %B1000000000000001,
set direction 0,
eval,
output;

// CASO 2:
// direction = 1 -> corrimiento a la DERECHA
// in = 1000 0000 0000 0001 (0x8001 = 32769 decimal)
// out = 0100 0000 0000 0000 (0x4000 = 16384 decimal)
// result = 1 (in[0] = 1, el Bit Menos significativo sale por la
//

set in %B1000000000000001,
set direction 1,
eval,
output;

```

Hardware Simulator (2.5) - C:\Users\Usuario\OneDrive\Documents\Usabella\4_SEMESTRE\Organizacion_de_computadores\Pruebas del proyecto 2\Shifter.hdl

File View Run Help

Chip Name: **Shifter** Time: 0

Input pins		Output pins	
Name	Value	Name	Value
in[16]	1	out[16]	2
direction	0	result	0

HDL

```

// direction = 1 -> corrimiento a la DERECHA
// in = 1000 0000 0000 0001 (0x8001 = 32769 decimal)
// out = 0100 0000 0000 0000 (0x4000 = 16384 decimal)
// result = 1 (in[0] = 1, el Bit Menos significativo sale por la
//

set in %B1000000000000001,
set direction 1,
eval,
output;

// Caso 3: Corrimiento izquierda con 0x0001 (decimal 1) -> out=0x000
set in %B0000000000000001,
set direction 0,
eval,
output;

```

6. Video explicativo

<https://youtu.be/Q1Jp9Q8Ld4Q?feature=shared>

Unidad Aritmético Lógica (ALU) con Shifter Integrado

1. Descripción General

El chip ALU (Arithmetic Logic Unit) es un componente fundamental dentro de la arquitectura del computador, encargado de ejecutar operaciones aritméticas y lógicas sobre dos operandos de 16 bits (x e y).

Esta implementación sigue el modelo clásico del proyecto Nand2Tetris, pero introduce una extensión significativa: la integración de un módulo Shifter, permitiendo realizar desplazamientos lógicos dentro de la misma unidad.

La ALU es capaz de:

- Ejecutar operaciones aritméticas (suma, resta, incremento, decremento)
- Ejecutar operaciones lógicas (AND, OR, NOT)
- Generar constantes (0, 1, -1)
- Evaluar condiciones (zr, ng)
- Realizar desplazamientos (shift left / shift right)
- Retornar el bit desplazado mediante la señal result

2. Interfaz del Componente

Señal	Ancho	Función
x[16]	16	Primer operando (base para desplazamientos).
y[16]	16	Segundo operando.
zx, nx	1	Control de x: Forzar a cero y negar bit a bit, respectivamente.

zy, ny	1	Control de y: Forzar a cero y negar bit a bit, respectivamente.
f	1	Selector de función: 0 para AND lógico, 1 para ADD aritmético. Además nos dice el shifter si es la derecha o izquierda.
no	1	Selector de negación de salida: Niega el resultado final si es 1. Solo para la ALU.

Señal	Ancho	Función
out[16]	16	Resultado final de la operación.
zr	1	Zero Flag: 1 si out == 0, de lo contrario 0.
ng	1	Negative Flag: 1 si out < 0, de lo contrario 0.
result	1	Bit de Desplazamiento: Bit expulsado durante el modo Shifter.

3. Descripción Funcional e Implementación del Circuito

El comportamiento de la ALU está completamente determinado por las señales de control (zx, nx, zy, ny, f, no), las cuales permiten modificar los operadores y seleccionar la operación a ejecutar. A nivel de implementación, el diseño se

estructura en varias etapas claramente definidas que operan de manera combinacional y en paralelo.

3.1 Preprocesamiento de entradas

Antes de realizar cualquier operación, los operandos x e y pasan por una etapa de acondicionamiento controlada por las señales:

- $zx = 1 \rightarrow$ fuerza $x = 0$
- $nx = 1 \rightarrow$ niega x bit a bit
- $zy = 1 \rightarrow$ fuerza $y = 0$
- $ny = 1 \rightarrow$ niega y bit a bit

Esto se implementa mediante una combinación de compuertas Mux16 y Not16, permitiendo construir múltiples valores derivados sin necesidad de circuitos adicionales.

3.2 Operaciones aritmético-lógicas

Una vez procesadas las entradas (xf y yf), la ALU calcula en paralelo dos posibles operaciones:

- Si $f = 0 \rightarrow$ se realiza la operación lógica: $xf \text{ AND } yf$
- Si $f = 1 \rightarrow$ se realiza la operación aritmética: $xf + yf$

Ambos resultados se generan simultáneamente ($andOut$ y $addOut$) y luego se seleccionan mediante un Mux16 controlado por f .

Posteriormente, si $no = 1$, el resultado seleccionado se niega bit a bit. Esta negación constituye la última etapa de la rama aritmético-lógica, permitiendo obtener operaciones como complemento, resta y otras derivadas.

3.3 Modo Shifter integrado

La ALU incorpora un modo especial que permite realizar desplazamientos directamente sobre el operando x , extendiendo su funcionalidad más allá de las operaciones tradicionales.

Activación del modo Shifter

Este modo se activa únicamente cuando se cumple la siguiente condición:

- $zx = 0, nx = 0, zy = 0, ny = 0$ y $no = 1$

Formalmente: $isShift = (NOT\ zx) \text{ AND } (NOT\ nx) \text{ AND } (NOT\ zy) \text{ AND } (NOT\ ny) \text{ AND } no$

Esta condición actúa como una firma de activación, garantizando que el modo Shifter no interfiera con las operaciones estándar de la ALU.

3.4 Operaciones de desplazamiento

Cuando $isShift = 1$, la ALU ignora completamente la lógica aritmético-lógica y selecciona el resultado proveniente del módulo Shifter, el cual opera directamente sobre la entrada x .

La señal f se reutiliza para determinar la dirección del desplazamiento:

- $f = 0 \rightarrow$ desplazamiento a la izquierda
- $f = 1 \rightarrow$ desplazamiento a la derecha

Desplazamiento a la izquierda:

- $out[i] = x[i-1]$
- $out[0] = 0$
- $result = x[15]$

Desplazamiento a la derecha:

- $out[i] = x[i+1]$
- $out[15] = 0$
- $result = x[0]$

Este diseño permite integrar eficientemente el corrimiento sin duplicar lógica dentro de la ALU.

3.5 Selección final de salida

Para determinar la salida final del sistema, se utiliza un Mux16 que selecciona entre:

- aluFinal: resultado de la rama aritmético-lógica (incluyendo negación)
- shiftOut: resultado del módulo Shifter

$out = Mux(aluFinal, shiftOut, isShift)$

Es importante destacar que la señal **no actúa como negación en modo Shifter**, sino como parte de la condición de activación, por lo que el resultado del desplazamiento no se ve afectado por esta señal.

3.6 Cálculo de flags

La ALU genera dos señales de estado fundamentales:

Flag negativo (ng) -> Se obtiene directamente del bit más significativo: $ng = out[15]$

Indica si el resultado es negativo en representación de complemento a dos.

Flag cero (zr) -> Se calcula verificando si todos los bits de salida son cero:

1. Se realiza un OR sobre los 8 bits bajos
2. Se realiza un OR sobre los 8 bits altos
3. Se combinan ambos resultados
4. Se niega el resultado final

$zr = NOT(OR(total\ bits\ de\ out))$

3.7 Cálculo de la salida result

La señal result permite capturar el bit que se pierde durante el desplazamiento:

- Si $isShift = 0$ -> $result = 0$
- Si $isShift = 1$ -> $result = shiftResult$

Implementación: $result = Mux(false, shiftResult, isShift)$

Esto asegura que la salida result solo tenga significado en el contexto del modo Shifter.

4. Casos de prueba

Operación / Caso	Entrada x[16]	Entrada y[16]	Control zx,nx,zy,ny,f,no	Salida out[16]	zr	ng	result
Shift Left	1000 0000 0000 0001	0000 0000 0000 0000	0 0 0 0 0 1	0000 0000 0000 0010	0	0	1
Shift Right	1000 0000 0000 0001	0000 0000 0000 0000	0 0 0 0 1 1	0100 0000 0000 0000	0	0	1
Suma (x+y)	0000 0000 0000 0101	0000 0000 0000 0010	0 0 0 0 1 0	0000 0000 0000 0111	0	0	0
Negación (!x)	0000 0000 0000 1010	1111 1111 1111 1111	0 0 1 1 0 1	1111 1111 1111 0101	0	1	0

Hardware Simulator (2.5) - C:\Users\Usuario\OneDrive\Documents\Isabella\4_SEMESTRE\Organizacion_de_computadores\Pruebas del proyecto 2\ALU.hdl

File View Run Help

Chip Name : **ALU** Time : 0

Input pins		Output pins	
Name	Value	Name	Value
x[16]	-32767	out[16]	2
y[16]	0	zr	0
zx	0	ng	0
nx	0	result	1
zy	0		
ny	0		
f	0		
no	1		

```
// ALU.tst - Script de prueba para la ALU extendida con Shifter
// Proyecto 2 - Organización del Computador
// Autores: Isabella Cadavid e Isabella Ocampo

load ALU.hdl,
output-file ALU.out,
compare-to ALU.cmp,
output-list x%B1.16.1 y%B1.16.1 zx%B1.1.1 nx%B1.1.1 zy%B1.1.1 ny%B1.1.1

// --- TABLA 1 DEL PROFE (Modo Shifter: NO se niega) ---

// Caso 1: Shift Left, x = 1000000000000001 (-32767)
// Esperado: out = 0000000000000010 (2), result = 1
set x %B1000000000000001, set y %B0000000000000000,
set zx 0, set nx 0, set zy 0, set ny 0, set f 0, set no 1,
eval, output;
```

Hardware Simulator (2.5) - C:\Users\Usuario\OneDrive\Documents\Isabella\4_SEMESTRE\Organizacion_de_computadores\Pruebas del proyecto 2\ALU.hdl

File View Run Help

Chip Name : **ALU** Time : 0

Input pins		Output pins	
Name	Value	Name	Value
x[16]	-32767	out[16]	16384
y[16]	0	zr	0
zx	0	ng	0
nx	0	result	1
zy	0		
ny	0		
f	1		
no	1		

```
compare-to ALU.cmp,
output-list x%B1.16.1 y%B1.16.1 zx%B1.1.1 nx%B1.1.1 zy%B1.1.1 ny%B1.1.1

// --- TABLA 1 DEL PROFE (Modo Shifter: NO se niega) ---

// Caso 1: Shift Left, x = 1000000000000001 (-32767)
// Esperado: out = 0000000000000010 (2), result = 1
set x %B1000000000000001, set y %B0000000000000000,
set zx 0, set nx 0, set zy 0, set ny 0, set f 0, set no 1,
eval, output;

// Caso 2: Shift Right, x = 1000000000000001 (-32767)
// Esperado: out = 0100000000000000 (16384), result = 1
set x %B1000000000000001, set y %B0000000000000000,
set zx 0, set nx 0, set zy 0, set ny 0, set f 1, set no 1,
eval, output;
```

Hardware Simulator (2.5) - C:\Users\Usuario\OneDrive\Documents\Isabella\4_SEMESTRE\Organizacion_de_computadores\Pruebas del proyecto 2\ALU.hdl

File View Run Help

Chip Name : **ALU** Time : 0

Input pins		Output pins	
Name	Value	Name	Value
x[16]	5	out[16]	7
y[16]	2	zr	0
zx	0	ng	0
nx	0	result	0
zy	0		
ny	0		
f	1		
no	0		

```
compare-to ALU.cmp,
output-list x%B1.16.1 y%B1.16.1 zx%B1.1.1 nx%B1.1.1 zy%B1.1.1 ny%B1.1.1

// --- TABLA 1 DEL PROFE (Modo Shifter: NO se niega) ---

// Caso 1: Shift Left, x = 1000000000000001 (-32767)
// Esperado: out = 0000000000000010 (2), result = 1
set x %B1000000000000001, set y %B0000000000000000,
set zx 0, set nx 0, set zy 0, set ny 0, set f 0, set no 1,
eval, output;

// Caso 2: Shift Right, x = 1000000000000001 (-32767)
// Esperado: out = 0100000000000000 (16384), result = 1
set x %B1000000000000001, set y %B0000000000000000,
set zx 0, set nx 0, set zy 0, set ny 0, set f 1, set no 1,
eval, output;

// --- 5 CASOS EXTRA (Mezcla ALU y Shifter) ---

// Extra 1: ALU Suma (x=5, y=2)
// Esperado: out = 0000000000000111 (7), result = 0
set x %B0000000000000101, set y %B0000000000000010,
set zx 0, set nx 0, set zy 0, set ny 0, set f 1, set no 0,
eval, output;
```

HDL		Internal pins	
Name		Value	
* ALU.hdl - ALU (Arithmetic Logic Unit)		xz[16]	
		5	

5. Video Explicativo

<https://youtu.be/OEYjY9LSjQg>

Arquitectura de la Instrucción para Operaciones Shift

1.1 Formato de la instrucción tipo C

Las instrucciones tipo C en la arquitectura Hack mantienen el formato de 16 bits:

111 a cccccc ddd jjj

- 111: identifica instrucción tipo C
- a: selecciona entre registro A o memoria M
- cccccc: define la operación de la ALU
- ddd: indica el destino del resultado
- jjj: define condiciones de salto

1.2 Problema identificado

La arquitectura original **no soporta operaciones de desplazamiento** (<<, >>).

Dado que todas las operaciones pasan por el campo comp, era necesario extender su codificación **sin modificar el formato de la instrucción**.

1.3 Solución propuesta

Se reutilizan combinaciones del campo comp para representar operaciones shift.

Reglas de codificación:

Campo / Bit	Valor	Función / Comportamiento
c1	0	El desplazamiento se aplica sobre el registro D .
	1	El desplazamiento se aplica sobre el registro A o la memoria M .

c2 - c5	1000	Código estático que identifica de forma única la operación de desplazamiento.
c6	0	Ejecuta un desplazamiento a la izquierda (<<).
	1	Ejecuta un desplazamiento a la derecha (>>).
Bit a	0	Utiliza el valor del registro A .
	1	Utiliza el valor de la memoria M .

1.4 Traducción de instrucciones

Operación	Código
D << 1	111 0 010000 ddd jjj
A << 1	111 0 110000 ddd jjj
M << 1	111 1 110000 ddd jjj
D >> 1	111 0 010001 ddd jjj
A >> 1	111 0 110001 ddd jjj

M >> 1	111 1 110001 ddd jjj
--------	----------------------

1.5 Ejemplos

- $D = M \ll 1 \rightarrow 1111110000010000$
- $AM = D \gg 1 \rightarrow 1110010001101000$

1.6 Conclusión

Esta solución permite agregar operaciones de desplazamiento reutilizando el campo comp, sin cambiar el formato original de la instrucción. Así, se amplía la funcionalidad de la ALU con cambios mínimos en la CPU.

Memoria

1. Descripción General

El chip Memory es el encargado de administrar todo el espacio de almacenamiento de datos del computador Hack. A diferencia de la ROM (que guarda las instrucciones), este chip maneja la memoria RAM y la entrada/salida.

Básicamente, este componente divide las direcciones disponibles para controlar tres partes:

- **RAM16K:** Guarda datos y variables del programa.
- **Screen (Pantalla):** Guarda bits que se dibujan en el monitor.
- **Keyboard (Teclado):** Lee la tecla que el usuario está presionando.

2. Interfaz del Componente

Señal	Ancho	Función
in[16]	16	El dato o número que queremos guardar en la memoria.
address[15]	15	La dirección exacta a la que queremos acceder.
load	1	Señal de permiso: 1 para escribir/guardar, 0 para solo leer.
out[16]	16	El dato que sale de la memoria hacia la CPU.

3. Descripción Funcional e Implementación

El diseño de la memoria funciona como un sistema de distribución. Usa los dos bits más significativos de la dirección (address[13] y address[14]) para saber a qué lugar enviar o pedir la información.

3.1 Combinaciones

- **00 y 01:** Van a la RAM16K (Ocupa el 50% de la memoria, de la dirección 0 a la 16383).
- **10:** Va a la Pantalla (Direcciones 16384 a 24575).
- **11:** Va al Teclado (Solo la dirección 24576). *(Nota: El bit 15 no entra a la memoria porque la CPU ya lo usó para saber que era una instrucción tipo A).*

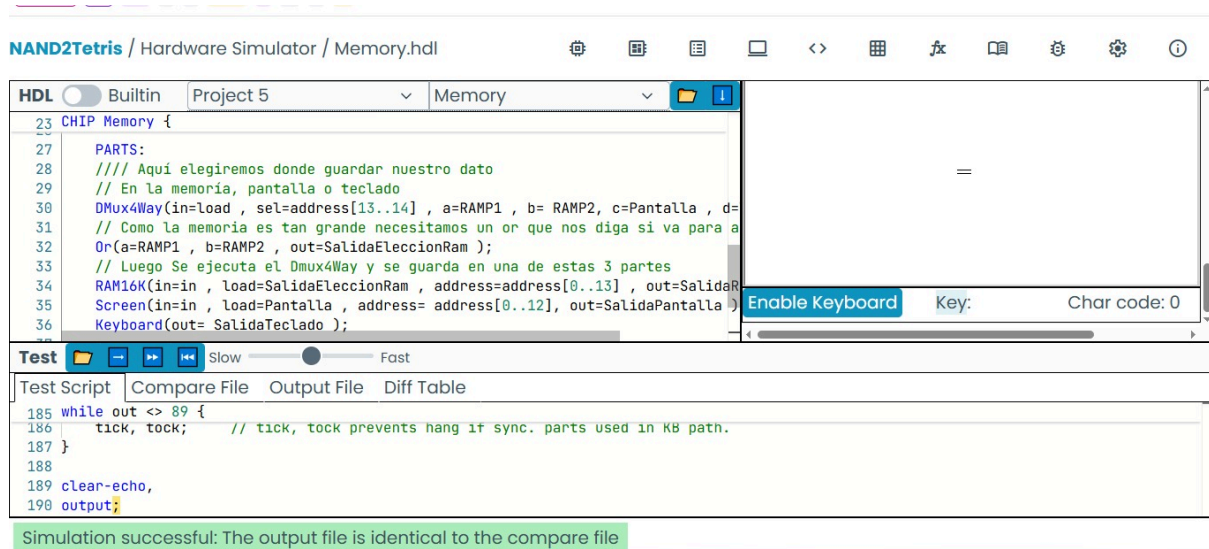
3.2 Proceso de Escritura (DMux4Way)

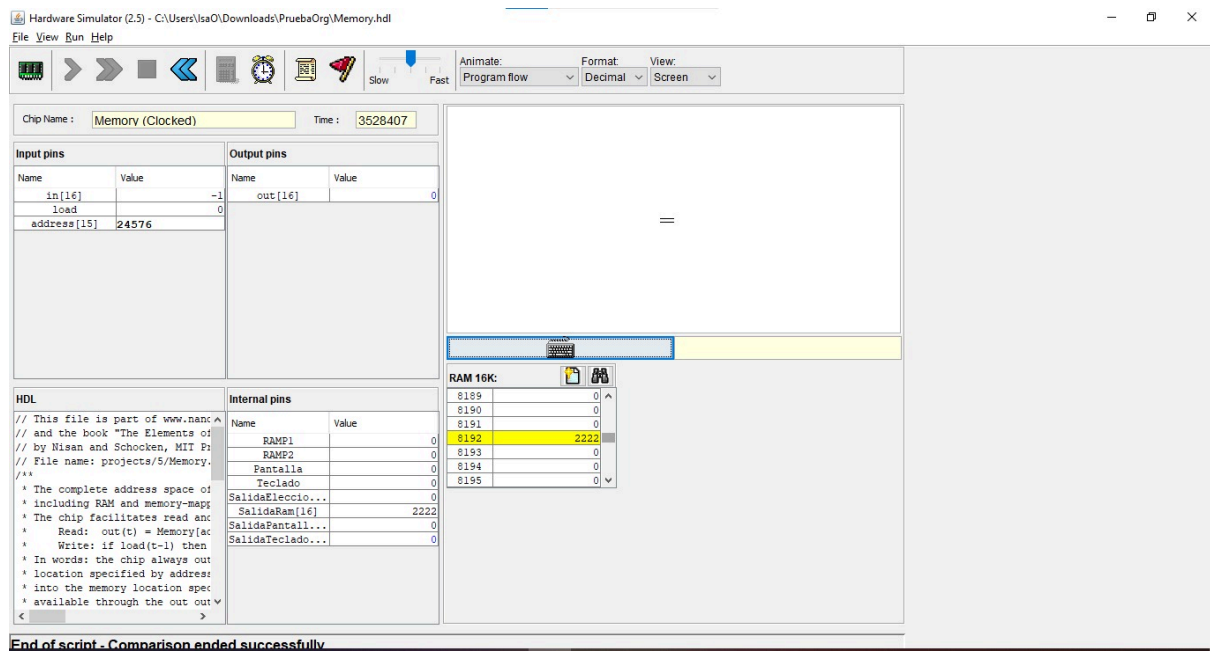
Para decidir a quién le damos el permiso de escribir (load), usamos un DMux4Way. Como este chip divide las señales en cuatro partes iguales, y nuestra RAM ocupa dos de esas partes, conectamos las salidas a y b del DMux a una compuerta Or. Si la señal se va por a o por b, el Or lo lleva hacia la RAM. La salida c va directo a la pantalla, y la salida d se ignora porque al teclado no se le puede escribir nada.

3.3 Proceso de Lectura (Mux4Way16)

Para leer, usamos la lógica en inverso. Un Mux4Way16 recibe los datos que salen de la RAM (conectada a los cables a y b), de la Pantalla (vía c) y del Teclado (vía d). Dependiendo de la dirección solicitada, el Mux elige cuál de esas respuestas dejar salir por el cable principal (out).

4. Pruebas





5. Video Explicativo

<https://www.youtube.com/watch?v=mq-yS4MR1L0>

Unidad Central de Procesamiento (CPU)

1. Descripción General

La CPU es el cerebro del computador. Su trabajo es leer las instrucciones de 16 bits que vienen de la ROM, decodificarlas, realizar los cálculos necesarios usando la ALU, y decidir si guarda los resultados o si salta a otra línea del código. Esta CPU está adaptada para integrar la ALU modificada (que incluye el Shifter), asegurando que todas las señales de control fluyan correctamente.

2. Interfaz del Componente

Señal	Ancho	Función
inM[16]	16	Dato que llega desde la memoria RAM.
instruction[16]	16	La instrucción de código que llega desde la ROM.
reset	1	Si es 1, reinicia el programa (manda el PC a 0).
outM[16]	16	Resultado que se va a enviar para guardar en la RAM.
writeM	1	Permiso de escritura para la RAM (1 = guardar).
addressM[15]	15	Dirección de la RAM a la que queremos acceder.
pc[15]	15	Dirección de la próxima instrucción que la ROM debe leer.

3. Descripción Funcional e Implementación

El funcionamiento se divide en la decodificación, el manejo de los registros internos, el cálculo en la ALU y la lógica de saltos.

3.1 Decodificación de la instrucción

Se evalúa el bit más significativo (instruction[15]). Si es 0, es una instrucción Tipo A (cargar una dirección). Si es 1, es una instrucción Tipo C (operación matemática).

3.2 Registros A y D

- **Registro A:** Guarda direcciones de memoria o constantes. Se actualiza si la instrucción es Tipo A, o si es Tipo C y el bit de destino d1 (instruction[5]) está en 1. Un Mux16 decide si entra la instrucción original o el resultado de la ALU.
- **Registro D:** Guarda únicamente datos de cálculos. Solo se actualiza si es una instrucción Tipo C y el bit de destino d2 (instruction[4]) está en 1.

3.3 Integración con la ALU y Shifter

La CPU utiliza la ALU personalizada. Un Mux16 controlado por el bit a (instruction[12]) decide si la ALU va a operar usando el Registro A o el dato que viene de la memoria (inM).

Se le pasan directamente los bits de control (c1 a c6) a los pines zx, nx, zy, ny, f, no.

Como la arquitectura oficial de la CPU Hack no utiliza el pin extra result generado por el Shifter de la ALU, este se conecta a un cable interno que se ignora, manteniendo la compatibilidad perfecta.

3.4 Program Counter (PC) y Saltos

El PC indica qué línea de código sigue. La lógica de salto combina los bits j1, j2, j3 de la instrucción con las banderas zr (cero) y ng (negativo) que escupe la ALU.

- Si la condición de salto se cumple (y es una instrucción Tipo C), el PC carga la dirección que está guardada en el Registro A.
- Si no hay salto, el PC simplemente incrementa su valor en 1 para pasar a la siguiente línea.

4. Pruebas

NAND2Tetris / Hardware Simulator / CPU.hdl

HDL Built-in Project 5 CPU Chip CPU Eval Reset Clock: 46

```

CHIP CPU {
33 // writeM para habilitar guardado en la memoria RAM (d3)
34 And(a=esTipoC, b=instruction[3], out=writeM);
35 // Lógica de los saltos con el PC
36 // Primero sacar si es positivo estricto (>0)
37 Or(a=negativo, b=cero, out=menorOIgualACero);
38 Not(in=menorOIgualACero, out=positivo);
39 // Revisar los bits j1, j2, j3 con las banderas de la ALU
40 And(a=instruction[2], b=negativo, out=saltoMenor);
41 And(a=instruction[1], b=cero, out=saltoIgual);
42 And(a=instruction[0], b=positivo, out=saltoMayor);
43 Or(a=saltoMenor, b=saltoIgual, out=saltoMenorOIgual);
}

```

Input pins

inM 0 0 1 0 1 0 1 1 0 1 1 0 0

instruction 0 1 1 1 1 1 1 1 1 1 1 1 1

reset 0

Output pins

outM 0 0 0 0 0 0 0 0 0 0 0 0 0

Test CPU Slow Fast

Test Script Compare File Run File Diff Table

```

145 tick, output, tock, output;
146
147 set instruction %B0111111111111111, // @32767
148 set reset 0;
149 tick, output, tock, output;

```

Simulation successful: The output file is identical to the compare file

Hardware Simulator (2.5) - C:\Users\IsaO\Downloads\PruebaOrg2\CPU.hdl

File View Run Help

Chip Name: CPU (Clock) Time: 48

Input pins		Output pins	
Name	Value	Name	Value
inM[16]	11111	outM[16]	1
instruction[16]	32767	writeM	0
reset	0	addressM[15]	32767
		pc[15]	1

HDL		Internal pins	
Name	Value	Name	Value
esTipoA	1		
esTipoC	0		
ALUUnaciaA	0		
salidaALU[16]	1		
entradaA[16]	32767		
cargarA	1		
salidaA[16]	32767		
cargarD	0		
salidaD[16]	1		
entradaAOM[16]	11111		
cero	0		
negativo	0		
menorOIgual...	0		

```

set instruction %B1110001100000110, // D;JLE
tick, output, tock, output;

set instruction %B1110001100000111, // D;JMP
tick, output, tock, output;

set instruction %B1110111111010000, // D=1
tick, output, tock, output;

set instruction %B1110001100000001, // D;JGT
tick, output, tock, output;

set instruction %B1110001100000010, // D;JEQ
tick, output, tock, output;

set instruction %B1110001100000011, // D;JGE
tick, output, tock, output;

set instruction %B1110001100000100, // D;JLT
tick, output, tock, output;

set instruction %B1110001100000101, // D;JNE
tick, output, tock, output;

set instruction %B1110001100000110, // D;JLE
tick, output, tock, output;

set instruction %B1110001100000111, // D;JMP
tick, output, tock, output;

set reset 1;
tick, output, tock, output;

set reset 0;
tick, output, tock, output;

```

End of script - Comparison ended successfully

Hardware Simulator (2.5) - C:\Users\IsaO\Downloads\PruebaOrg2\CPU.hdl

File View Run Help

Chip Name: CPU (Clocked) Time: 48

Input pins		Output pins	
Name	Value	Name	Value
inM[16]	11111	outM[16]	1
instruction[16]	32767	writeM	0
reset	0	addressM[15]	32767
		pc[15]	1

Internal pins	
Name	Value
esTipoA	1
esTipoC	0
ALU hacia A	0
salidaALU[16]	1
entradaA[16]	32767
cargarA	1
salidaA[16]	32767
cargarD	0
salidaD[16]	1
entradaAOM[16]	11111
cero	0
negativo	0
menorO Igual...	0
positivo	1

```

HDL
/*****
 * CPU.hdl
 * Autor 1: Isabella Ocampo SÃ¡nchez
 * Autor 2: Isabella Cadavid Posada
 *****/

CHIP CPU {
  IN inM[16], instruction[16]
  OUT outM[16], writeM, addressM[15], pc[15]
  PARTS:
    // Debemos saber si es una instrucciÃ³n de tipo A o C
    Not(in=instruction[15], out=esTipoA)
    Not(in=esTipoA, out=esTipoC)
    // Si es tipo C y donde guardamos el resultado
    set instruction %B1110001100000110, // D;JLE
    tick, output, tock, output;

    set instruction %B1110001100000111, // D;JMP
    tick, output, tock, output;

    set instruction %B110111111010000, // D=1
    tick, output, tock, output;

    set instruction %B1110001100000001, // D;JGT
    tick, output, tock, output;

    set instruction %B1110001100000010, // D;JEQ
    tick, output, tock, output;

    set instruction %B1110001100000011, // D;JGE
    tick, output, tock, output;

    set instruction %B1110001100000100, // D;JLT
    tick, output, tock, output;

    set instruction %B1110001100000101, // D;JNE
    tick, output, tock, output;

    set instruction %B1110001100000110, // D;JLE
    tick, output, tock, output;

    set instruction %B1110001100000111, // D;JMP
    tick, output, tock, output;

    set reset 1;
    tick, output, tock, output;

    set instruction %B0111111111111111, // @32767
    set reset 0;
    tick, output, tock, output;
}

```

End of script - Comparison ended successfully

5. Video Explicativo

<https://www.youtube.com/watch?v=NrBvYcYYwWE>

Computador

1. Descripción General

El chip Computer es el nivel más alto de la arquitectura. Su única función es interconectar los tres grandes bloques del sistema: la memoria de instrucciones (ROM32K), el procesador (CPU) y la memoria de datos (Memory). No contiene compuertas lógicas adicionales, solo el cableado estructural.

2. Interfaz del Componente

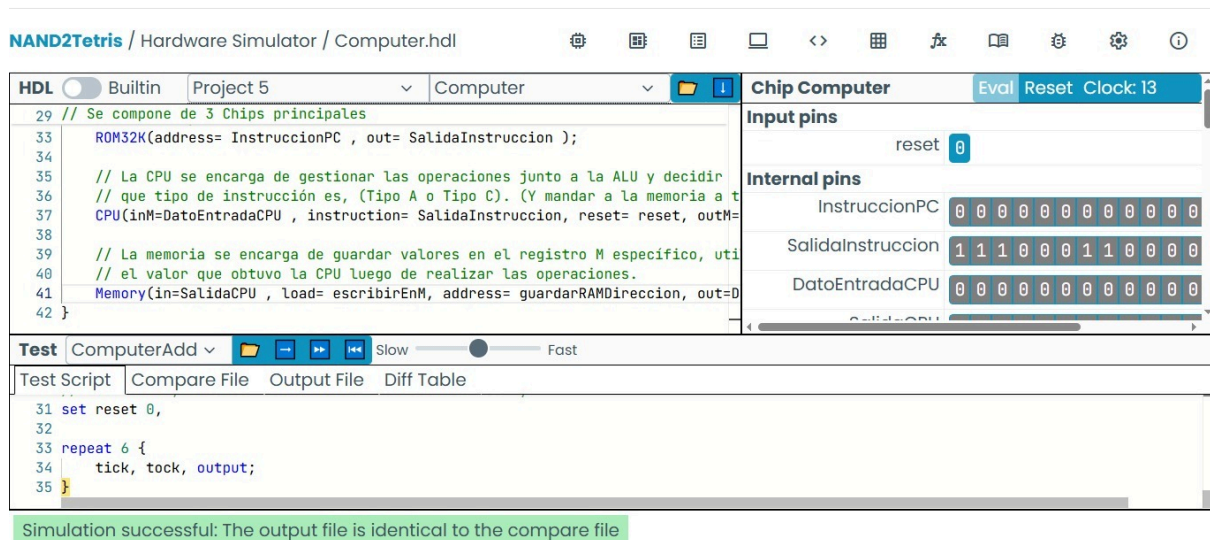
Señal	Ancho	Función
reset	1	Señal maestra. Si es 1, la CPU reinicia el programa leyendo desde la línea 0 de la ROM. Si es 0, el programa corre normalmente.

3. Descripción Funcional e Implementación

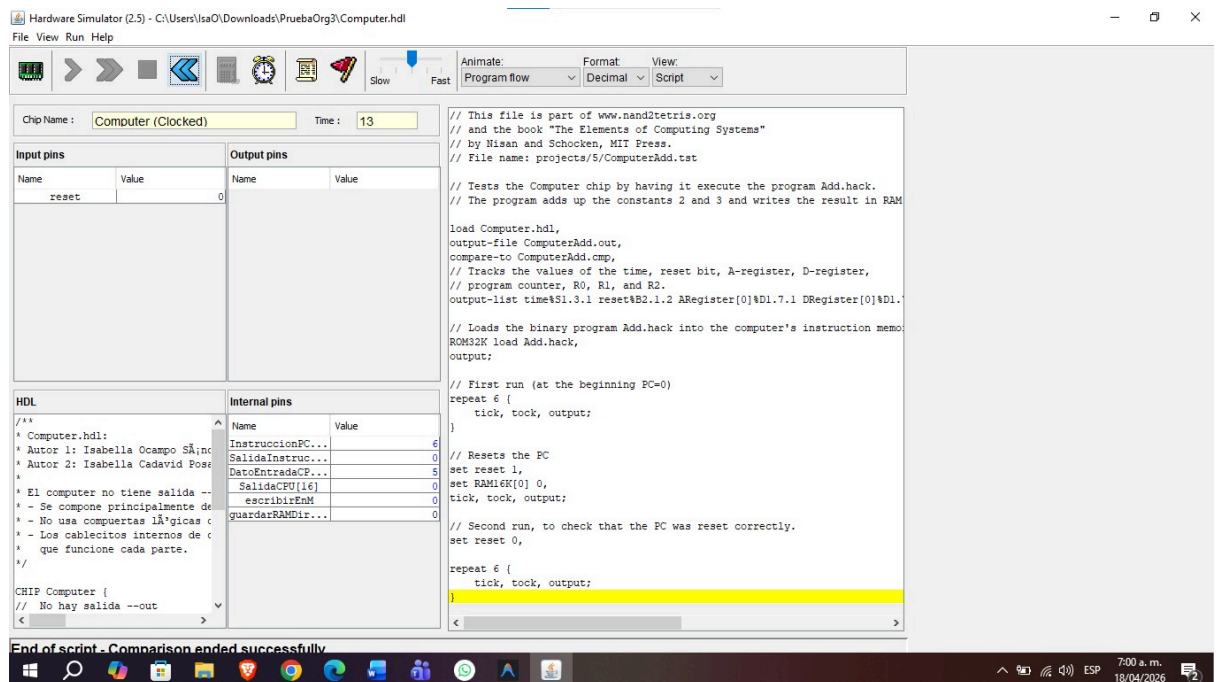
El diseño refleja el ciclo "Fetch-Execute" (Buscar y Ejecutar) clásico de las computadoras:

1. **Conexión de la ROM:** La ROM32K recibe el cable pc (Program Counter) desde la CPU. Con este número, busca la línea de código correspondiente y la envía hacia la CPU a través del cable de instrucción.
2. **Conexión de la CPU:** La CPU recibe la instrucción de la ROM y el dato actual de la Memoria (inM). Procesa la información y genera las salidas necesarias: un nuevo valor de pc, un dato para guardar (outM), una dirección (addressM) y el permiso de escritura (writeM).
3. **Conexión de la Memoria:** El chip Memory recibe las señales outM, addressM y writeM de la CPU. Dependiendo de los valores, guarda el dato en la RAM/Pantalla, o simplemente devuelve el dato solicitado hacia la CPU para el siguiente ciclo.

4. Pruebas

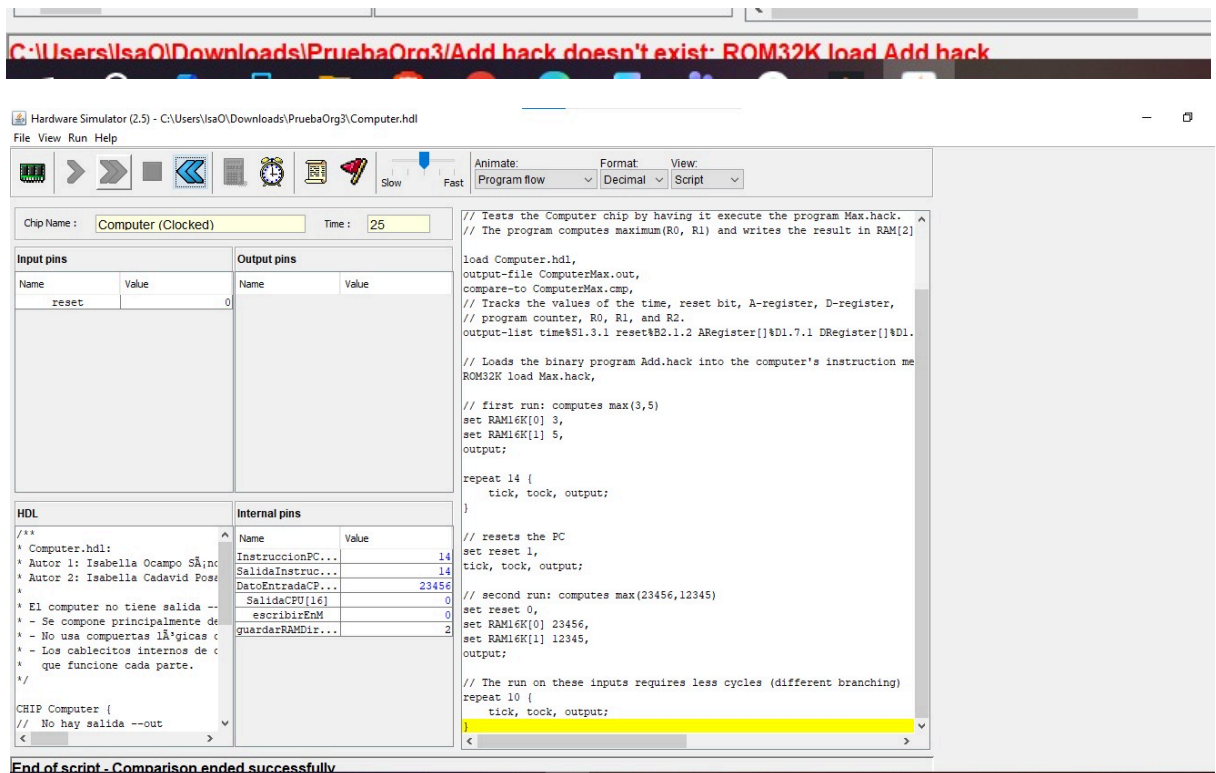


- **ComputerAdd:** Es la prueba más básica. Solo verifica que la CPU sepa leer números, pasarlos por la ALU para sumarlos y guardar el resultado. Específicamente, este programa suma los números 2 y 3 y guarda el resultado en el cajón RAM[0].

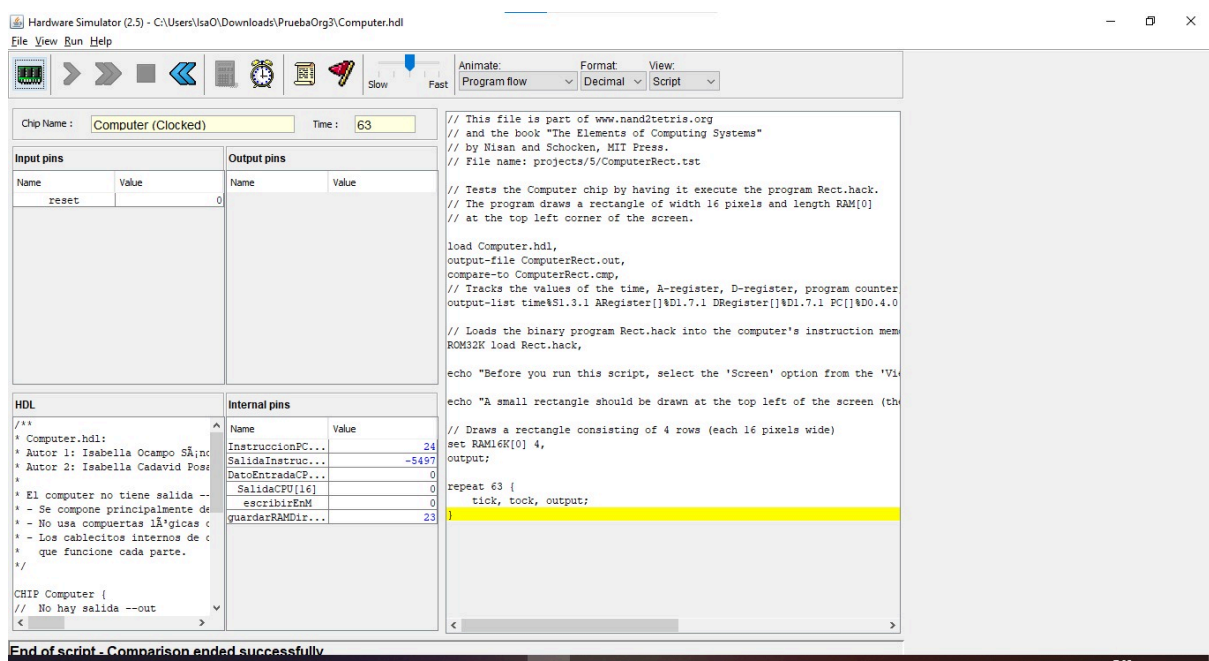


- **ComputerMax:** El programa evalúa dos números en RAM[0] y RAM[1], calcula cuál es el mayor, y lo guarda en RAM[2]. Para hacer esto, la CPU tiene que saber tomar decisiones (saltar a una parte u otra de la receta dependiendo de si un número es mayor que otro).

Pruebas de intento y error durante la ejecución:



- **ComputerRect:** Este programa es un ciclo complejo que interactúa con la pantalla. Dibuja un rectángulo de 16 píxeles de ancho en la esquina superior izquierda de la pantalla. Esto prueba que tu CPU sabe hacer ciclos (loops), usar punteros y, lo más importante, escribir correctamente en la zona de la memoria asignada a la pantalla (Screen).



5. Video Explicativo

<https://www.youtube.com/watch?v=8Qnr-G1wdPo>